

Data privacy and security

Midterm project presentation
Group 12

Experimenting with differential privacy in Deep
Learning

Background and Motivation

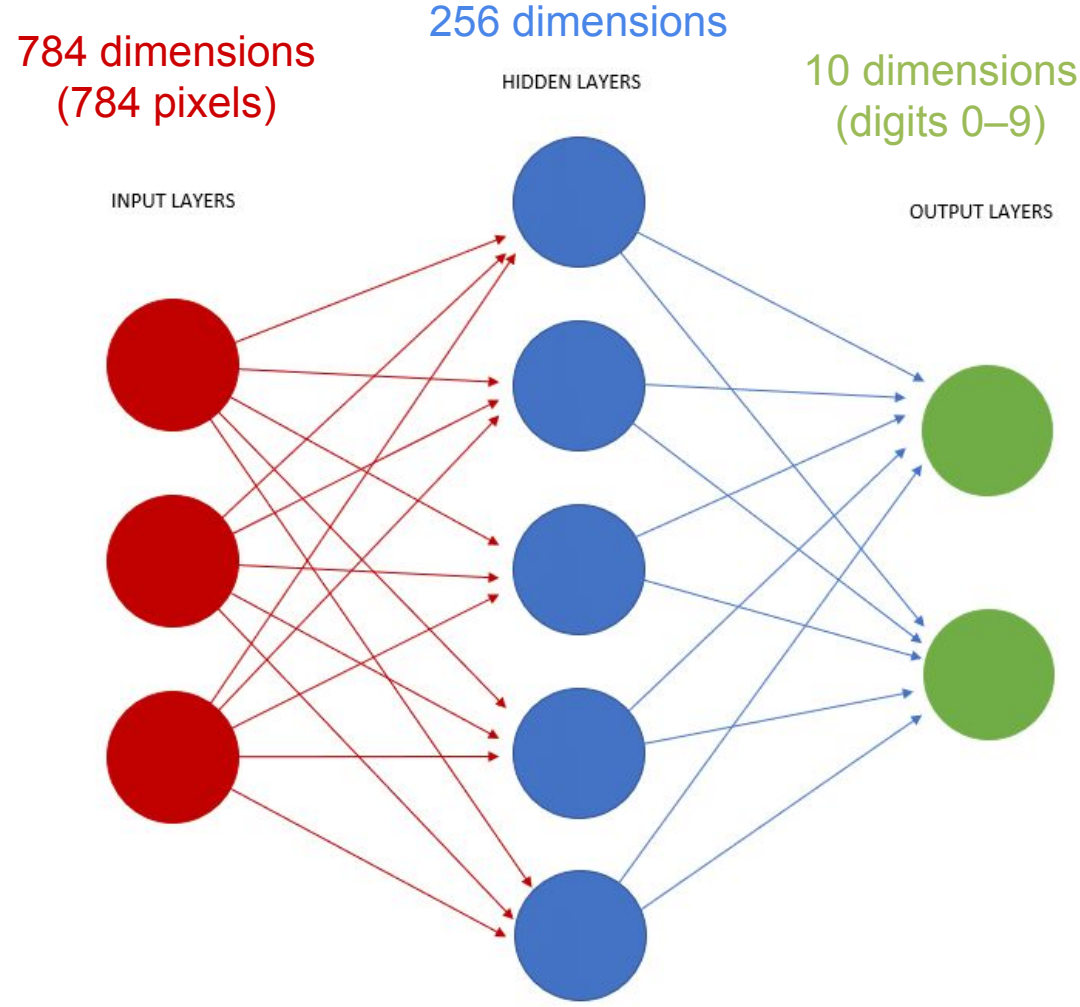
- Deep learning models have achieved success in a lot of applications.
 - But training of these models often requires access to sensitive data.
- Differential Privacy (DP) ensures outputs of an algorithm are statistically similar.
 - Achieved by adding calibrated noise to **training gradients** and **model outputs**.
- Metric: Privacy Budget (ϵ)
 - Smaller ϵ : stronger privacy but lower model accuracy.
- Use Opacus to apply DP in training
 - Goal: Evaluate the **trade-off between accuracy and privacy**

Methodology - DP Implementation

- Use DP-SGD via Opacus's PrivacyEngine
 - Clipping the per-sample gradients to a fixed norm (1.0 in our setup)
 - Adding Gaussian noise to the aggregated gradients
 - `get_epsilon()` used to track ϵ during training process.

Methodology - SimpleNN

- Input: Flattens 28×28 MNIST images
- Hidden Layer: 256 neurons, ReLU
- Dropout: rate = 0.3
- Output: 10 neurons for classification



Methodology - CNN

- Convolutional Layers
 - Conv1: 32 filters, 3×3 , padding=1
 - Conv2: 64 filters, 3×3 , padding=1
- Pooling
 - Max Pooling (2×2) → downsample to 14×14
- Dropout Regularization:
 - Dropout rate = 0.25
 - 2 dropout layer
- Fully Connected Layers
 - Flatten → 128 hidden units → Output (10 classes)

Experiment Setting

- Dataset: MNIST
 - 70,000 grayscale images of handwritten digits(training: 60,000, testing: 10,000)
 - Image size: 28×28 pixels
- Framework and Libraries
 - PyTorch: building and training neural networks
 - Opacus: integrate Differential Privacy into the PyTorch training loop.
 - TensorBoard: visualization of loss, accuracy, and ϵ .

Result with Simple NN

Training Setup

- Optimizer: DP-SGD (Differentially Private Stochastic Gradient Descent)
- Learning rate: 0.1 (with StepLR scheduler, gamma = 0.5 every 2 epochs)
- Noise multiplier: 1.1
- Batch size: 64
- Epochs: 10
- Clipping norm: 1.0

Result with Simple NN

Epoch	Avg Loss	Accuracy	Epsilon
1	0.8454	86.58%	0.14
2	0.7086	88.19%	0.19
3	0.7305	88.86%	0.23
4	0.7429	89.38%	0.27
5	0.7237	89.88%	0.30
6	0.6937	90.11%	0.33
7	0.6983	90.30%	0.36
8	0.6818	90.43%	0.38
9	0.6797	90.46%	0.40
10	0.6886	90.56%	0.43

Result with Simple NN

What happens without differential privacy?

- Full use of training data, means higher accuracy.
- But no privacy guarantees.
- Training in 10 epochs
 - Without DP: 98.24%, with DP: 90.56%

Result with CNN

Training Setup

- Optimizer: DP-SGD (Differentially Private Stochastic Gradient Descent)
- Learning rate: 0.1 (with StepLR scheduler, gamma = 0.5 every 2 epochs)
- Noise multiplier: 1.1
- Batch size: 64
- Epochs: 10
- Clipping norm: 1.0

Result with CNN

Epoch	Avg Loss	Accuracy	Epsilon
1	1.3453	87.30%	0.14
2	1.6027	88.64%	0.19
3	1.5222	89.33%	0.23
4	1.3659	89.10%	0.27
5	1.2708	90.33%	0.30
6	1.2008	90.30%	0.33
7	1.1743	90.68%	0.36
8	1.1141	90.86%	0.38
9	1.1077	91.12%	0.40
10	1.0691	91.13%	0.43

Result with CNN

What happens if the number of epochs is doubled?

- 20 Epochs for training with same DP-SGD configuration used
- Test Accuracy: 91.13% $\rightarrow$ 91.43%
- Privacy Budget (ϵ): From 0.43 $\rightarrow$ 0.61

Comparison of Simple NN and CNN

Model(epoch = 10)	Accuracy	Loss	Privacy Budget (ϵ)
Simple NN	90.56%	0.6886	0.43
CNN	91.13%	1.0691	0.43

- CNN shows **better accuracy but higher loss** under the same privacy constraints.
- ϵ evolved identically for both models.
 - Because ϵ is determined by training parameters, not by model architecture.

Conclusion

- A good tradeoff can be made losing less than 10% of accuracy while maintaining an epsilon value very low meaning excellent privacy.
- Depending on the risks carried and the need of precision, every model maker can make use of some differential privacy.